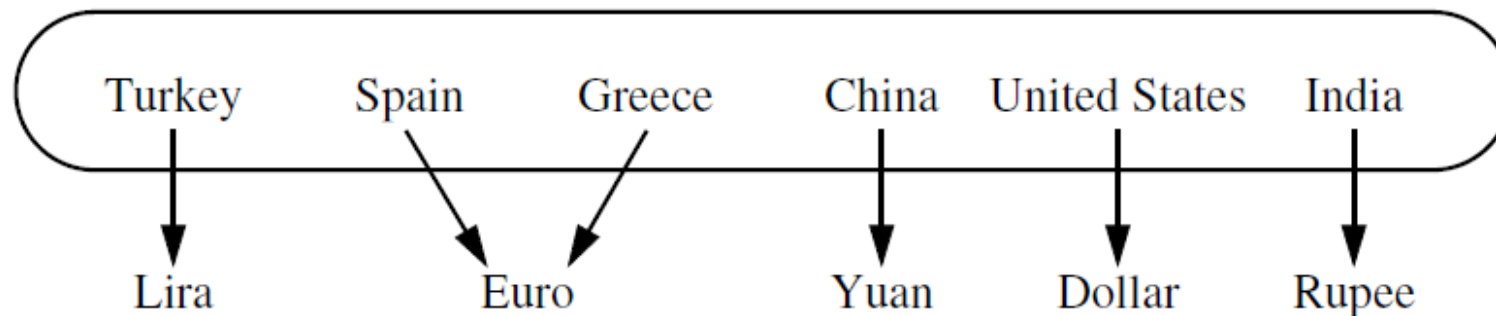


Dictionnaires

rouanehm@gmail.com

Dictionnaire

- Définition: un dictionnaire est un ensemble dynamique d'objets avec des clés comparables. Cette structure est aussi connue par 'Map' du terme mathématique 'mapping' qui signifie une fonction.
- Exemples d'application :
 - Table de symboles dans un compilateur (symbole $\leftrightarrow$ type)
 - Table de routage d'un DNS (adresse $\leftrightarrow$ chemin)



Dictionnaire en python : la classe dict

- En python, **dict** est la classe la plus significative du langage.
- <https://realpython.com/python-dicts/>

M[k]:	Return the value <i>v</i> associated with key <i>k</i> in map <i>M</i> , if one exists; otherwise raise a <code>KeyError</code> . In Python, this is implemented with the special method <code>__getitem__</code> .
M[k] = v:	Associate value <i>v</i> with key <i>k</i> in map <i>M</i> , replacing the existing value if the map already contains an item with key equal to <i>k</i> . In Python, this is implemented with the special method <code>__setitem__</code> .
del M[k]:	Remove from map <i>M</i> the item with key equal to <i>k</i> ; if <i>M</i> has no such item, then raise a <code>KeyError</code> . In Python, this is implemented with the special method <code>__delitem__</code> .
len(M):	Return the number of items in map <i>M</i> . In Python, this is implemented with the special method <code>__len__</code> .
iter(M):	The default iteration for a map generates a sequence of <i>keys</i> in the map. In Python, this is implemented with the special method <code>__iter__</code> , and it allows loops of the form, for k in M.

Operation	Return Value	Map
<code>len(M)</code>	0	<code>{ }</code>
<code>M['K'] = 2</code>	–	<code>{'K': 2}</code>
<code>M['B'] = 4</code>	–	<code>{'K': 2, 'B': 4}</code>
<code>M['U'] = 2</code>	–	<code>{'K': 2, 'B': 4, 'U': 2}</code>
<code>M['V'] = 8</code>	–	<code>{'K': 2, 'B': 4, 'U': 2, 'V': 8}</code>
<code>M['K'] = 9</code>	–	<code>{'K': 9, 'B': 4, 'U': 2, 'V': 8}</code>
<code>M['B']</code>	4	<code>{'K': 9, 'B': 4, 'U': 2, 'V': 8}</code>
<code>M['X']</code>	<code>KeyError</code>	<code>{'K': 9, 'B': 4, 'U': 2, 'V': 8}</code>
<code>M.get('F')</code>	<code>None</code>	<code>{'K': 9, 'B': 4, 'U': 2, 'V': 8}</code>
<code>M.get('F', 5)</code>	5	<code>{'K': 9, 'B': 4, 'U': 2, 'V': 8}</code>
<code>M.get('K', 5)</code>	9	<code>{'K': 9, 'B': 4, 'U': 2, 'V': 8}</code>
<code>len(M)</code>	4	<code>{'K': 9, 'B': 4, 'U': 2, 'V': 8}</code>
<code>del M['V']</code>	–	<code>{'K': 9, 'B': 4, 'U': 2}</code>
<code>M.pop('K')</code>	9	<code>{'B': 4, 'U': 2}</code>
<code>M.keys()</code>	<code>'B', 'U'</code>	<code>{'B': 4, 'U': 2}</code>
<code>M.values()</code>	4, 2	<code>{'B': 4, 'U': 2}</code>
<code>M.items()</code>	<code>('B', 4), ('U', 2)</code>	<code>{'B': 4, 'U': 2}</code>
<code>M.setdefault('B', 1)</code>	4	<code>{'B': 4, 'U': 2}</code>
<code>M.setdefault('A', 1)</code>	1	<code>{'A': 1, 'B': 4, 'U': 2}</code>
<code>M.popitem()</code>	<code>('B', 4)</code>	<code>{'A': 1, 'U': 2}</code>

Dictionnaire en python:

Compter la fréquence des mots d'un fichier

```
1 input_file = 'D:\\GDrive\\F20\\5P6.SDA\\TD\\SDAF20TD\\src\\datasets\\input_file.txt'
2 freq = {}
3 for piece in open(input_file).read().lower().split():
4     # only consider alphabetic characters within this piece
5     word = ''.join(c for c in piece if c.isalpha())
6     # print(f'Adding {word} ')
7     if word: # require at least one alphabetic character
8         freq[word] = 1 + freq.get(word, 0)
9
10 max_word = ''
11 max_count = 0
12 for (word, count) in freq.items():
13     # (key, value) tuples represent (word, count)
14     if count > max_count:
15         max_word = word
16         max_count = count
17 print(f'The most frequent word is {max_word}')
18 print(f'It's number of occurrences is {max_count}')
```

Dictionnaire en python :

Complexité de dict versus liste []

Operation	Example	Complexity Class	Notes
Index	<code>d[k]</code>	$O(1)$	
Store	<code>d[k] = v</code>	$O(1)$	
Length	<code>len(d)</code>	$O(1)$	
Delete	<code>del d[k]</code>	$O(1)$	
Containment	<code>k in/not in d</code>	$O(1)$	
Pop	<code>d.pop(k)</code>	$O(1)$	
Pop item	<code>d.popitem()</code>	$O(1)$	
Clear	<code>d.clear()</code>	$O(1)$	similar to <code>s = {}</code> or <code>= dict()</code>
View	<code>d.keys()</code>	$O(1)$	same for <code>d.values()</code>
Construction	<code>dict(...)</code>	$O(\text{len}(...))$	depends # (key,value) 2-tuples
Iteration	<code>for k in d:</code>	$O(N)$	all forms: keys, values, items

Operation	Example	Complexity Class	Notes
Index	<code>l[i]</code>	$O(1)$	
Store	<code>l[i] = 0</code>	$O(1)$	
Length	<code>len(l)</code>	$O(1)$	
Append	<code>l.append(5)</code>	$O(1)$	
Pop	<code>l.pop()</code>	$O(1)$	same as <code>l.pop(-1)</code> , popping at end
Clear	<code>l.clear()</code>	$O(1)$	similar to <code>l = []</code>
Slice	<code>l[a:b]</code>	$O(b-a)$	<code>l[1:5]:O(1)/l[:]:O(len(l)-0)=O(N)</code>
Extend	<code>l.extend(...)</code>	$O(\text{len}(...))$	depends only on len of extension
Construction	<code>list(...)</code>	$O(\text{len}(...))$	depends on length of ... iterable
check ==, !=	<code>l1 == l2</code>	$O(N)$	
Insert	<code>l[a:b] = ...</code>	$O(N)$	
Delete	<code>del l[i]</code>	$O(N)$	
Containment	<code>x in/not in l</code>	$O(N)$	searches list
Copy	<code>l.copy()</code>	$O(N)$	Same as <code>l[:]</code> which is $O(N)$
Remove	<code>l.remove(...)</code>	$O(N)$	
Pop	<code>l.pop(i)</code>	$O(N)$	$O(N-i)$: <code>l.pop(0):O(N)</code> (see above)
Extreme value	<code>min(l)/max(l)</code>	$O(N)$	searches list
Reverse	<code>l.reverse()</code>	$O(N)$	
Iteration	<code>for v in l:</code>	$O(N)$	
Sort	<code>l.sort()</code>	$O(N \log N)$	key/reverse mostly doesn't change
Multiply	<code>k*l</code>	$O(k N)$	<code>5*l</code> is $O(N)$: <code>len(l)*l</code> is $O(N**2)$

<https://tp60011275.blogspot.com/2018/12/performance-of-python-data-structures.html>

ADT Dictionnaire

- Les opérations les plus populaires d'un dictionnaire sont set/put, get et remove.
- Dans beaucoup de langage on peut définir l'opérateur []

`size()`: Returns the number of entries in M .

`isEmpty()`: Returns a boolean indicating whether M is empty.

`get( $k$ )`: Returns the value v associated with key k , if such an entry exists; otherwise returns null.

`put( $k$ ,  $v$ )`: If M does not have an entry with key equal to k , then adds entry (k, v) to M and returns null; else, replaces with v the existing value of the entry with key equal to k and returns the old value.

`remove( $k$ )`: Removes from M the entry with key equal to k , and returns its value; if M has no such entry, then returns null.

`keySet()`: Returns an iterable collection containing all the keys stored in M .

`values()`: Returns an iterable collection containing all the *values* of entries stored in M (with repetition if multiple keys map to the same value).

`entrySet()`: Returns an iterable collection containing all the key-value entries in M .

Exemple d'utilisation d'un dictionnaire

- Get : recherche
- Put : insertion
- Remove: suppression

Method	Return Value	Map
isEmpty()	true	{}
put(5,A)	null	{(5,A)}
put(7,B)	null	{(5,A), (7,B)}
put(2,C)	null	{(5,A), (7,B), (2,C)}
put(8,D)	null	{(5,A), (7,B), (2,C), (8,D)}
put(2,E)	C	{(5,A), (7,B), (2,E), (8,D)}
get(7)	B	{(5,A), (7,B), (2,E), (8,D)}
get(4)	null	{(5,A), (7,B), (2,E), (8,D)}
get(2)	E	{(5,A), (7,B), (2,E), (8,D)}
size()	4	{(5,A), (7,B), (2,E), (8,D)}
remove(5)	A	{(7,B), (2,E), (8,D)}
remove(2)	E	{(7,B), (8,D)}
get(2)	null	{(7,B), (8,D)}
remove(2)	null	{(7,B), (8,D)}
isEmpty()	false	{(7,B), (8,D)}
entrySet()	{(7,B), (8,D)}	{(7,B), (8,D)}
keySet()	{7, 8}	{(7,B), (8,D)}
values()	{B, D}	{(7,B), (8,D)}

Operation	C++ Map<keytype, valuetype>	Java HashMap<keytype, valuetype>	C# hashtable
get	Map[key]	Get(key)	Hash[key]
put	Insert(key, value)	Put(key, value)	Add(key, value)
containsKey	Count(key)	containsKey(key)	
removeKey	Erase(key)	Remove(key)	

Implémentation d'un dictionnaire

1. Implémentation par des structures de données simples :

1. Tableau

- Tableau avec les clés non trié
- Tableau avec les clés triés
- Tableau à accès direct
- Listes chaînées

2. Implémentation par des structures complexes :

1. Arbres binaires de recherche

2. Tables de hachage (prochain cours)

Implémentation d'un dictionnaire :

Un tableau non trié (1/2)

- Les éléments (couples clé-valeur) sont stockés dans un tableau. Chaque position du tableau contient un couple (k,v)
- Envisageable si on sait que la taille des ensembles (clés et valeurs) ne dépassera jamais la taille du tableau.
 - Avantage : simplicité
 - Inconvénients : opérations lentes.
- L'implantation par tableau ou par liste chaînée du dictionnaire demande, dans le pire cas, n opérations pour exécuter une insertion, une suppression ou une recherche. Complexité : $O(n)$.

Implémentation d'un dictionnaire : Un tableau non trié (1/2)

```
1 from src.adt.map.Entry import Entry
2
3
4 class UnsortedArrayMap(object):
5
6     def __init__(self):
7         self.__storage = []
8
9     def get(self, key):
10        for entry in self.__storage:
11            if key == entry.key:
12                return entry.value
13            raise KeyError('Key {repr(k)} not found')
14
15    def set(self, key, value):
16        for entry in self.__storage:
17            if key == entry.key:
18                entry.value = value
19                return
20        # did not find match for key
21        self.__storage.append(Entry(key, value))
```

```
4 class Entry(object):
5
6     def __init__(self, key, value, provided_hash=None):
7         self.key = key
8         self.value = value
9         self.hash = provided_hash
10
11    def __eq__(self, other):
12        # compare items based on their keys
13        return self.key == other.key
14
15    def __ne__(self, other):
16        return not (self == other)
17
18    def __lt__(self, other):
19        # compare items based on their keys
20        return self.key < other.key
21
22    def __str__(self):
23        return "Entry: key={0} value={1}>"\
24            .format(self.key, self.value)
```

```
23 def delete(self, key):
24     # Remove item associated with key k
25     for entry in range(len(self.__storage)):
26         if key == self.__storage[entry].key:
27             self.__storage.pop(entry)
28             return
29     raise KeyError('Key not found {repr(key)}')
30
31 def len(self):
32     return len(self.__storage)
33
34 def __iter__(self):
35     for entry in self.__storage:
36         yield entry.key
37
38 def __str__(self):
39     result = []
40     for entry in self.__storage:
41         result.append(str(entry))
42     to_return = '{}'.format(',\n'.join(result))
43     return '{' + to_return + '}'
```

Implémentation d'un dictionnaire :

Un tableau trié (1/2)

- Les éléments (couple composé de la clé et de la valeur) sont stockés dans un tableau. Les éléments sont triés selon la clé.
- Une recherche dichotomique (approche diviser-pour-régner) permet de réduire la complexité de la recherche ($O(\log n)$).

```
BINARY-SEARCH(V, k, low, high)
1  if low > high
2      return NIL
3  mid =  $\lfloor (\textit{low} + \textit{high}) / 2 \rfloor$ 
4  x = ELEM-AT-RANK(V, mid)
5  if k == x.key
6      return x
7  elseif k > x.key
8      return BINARY-SEARCH(V, k, mid + 1, high)
9  else return BINARY-SEARCH(V, k, low, mid - 1)
```

```
23 def binary_search_recursive(array, element, start, end):
24     if start > end:
25         return -1
26     mid = (start + end) // 2
27     if element == array[mid]:
28         return mid
29
30     if element < array[mid]:
31         return binary_search_recursive(array, element, start, mid-1)
32     else:
33         return binary_search_recursive(array, element, mid+1, end)
```

Implémentation d'un dictionnaire :

Un tableau trié (2/2)

- Insertion : recherche de la position par Binary-Search() puis insertion dans le vecteur par Insert-At-Rank qui nécessite un décalage des éléments vers la droite.
- Suppression : recherche puis suppression par Remove-At-Rank() qui nécessite un décalage des éléments vers la gauche.
- Complexité au pire cas :

<i>Implémentation</i>	SEARCH	INSERT	DELETE
Liste	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Vecteur trié	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$

Implémentation d'un dictionnaire :

Un tableau à accès direct

- Le dictionnaire est implémenté par un tableau de taille N fixée contenant les éléments de l'ensemble.
 - **Chaque position dans le tableau correspond a une clé (key=index)**
 - S'il y a une valeur x avec la clé k , alors $T[k]$ contient la valeur x , sinon $T[k]=NIL$

DIRECT-ADDRESS-SEARCH(T, k)

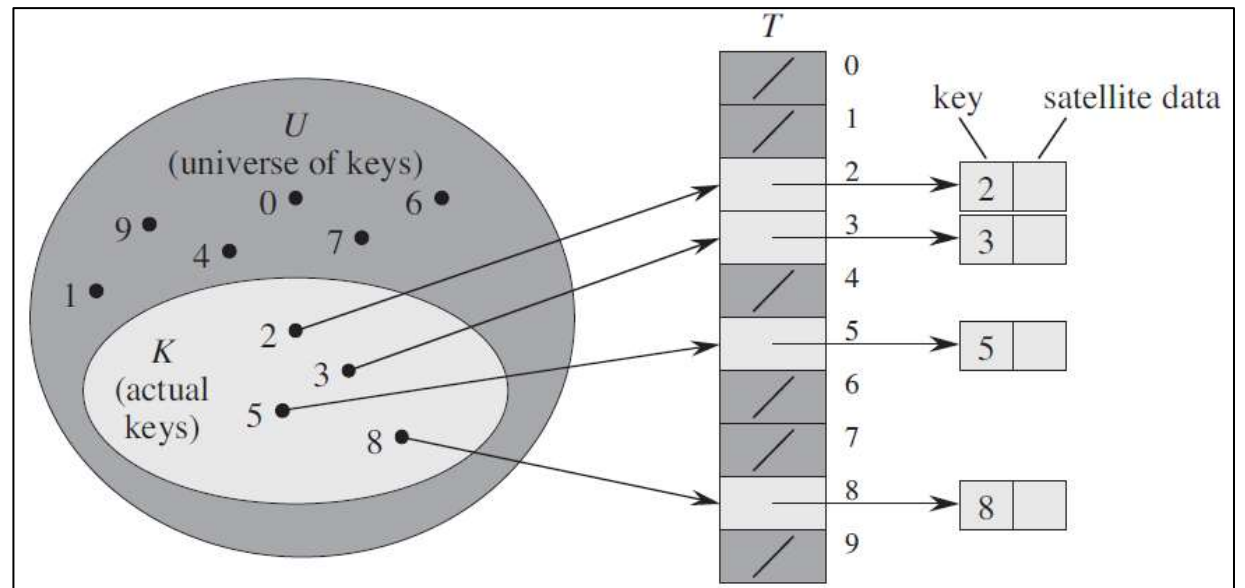
1 **return** $T[k]$

DIRECT-ADDRESS-INSERT(T, x)

1 **return** $T[x.key] = x$

DIRECT-ADDRESS-DELETE(T, x)

1 **return** $T[x.key] = NIL$



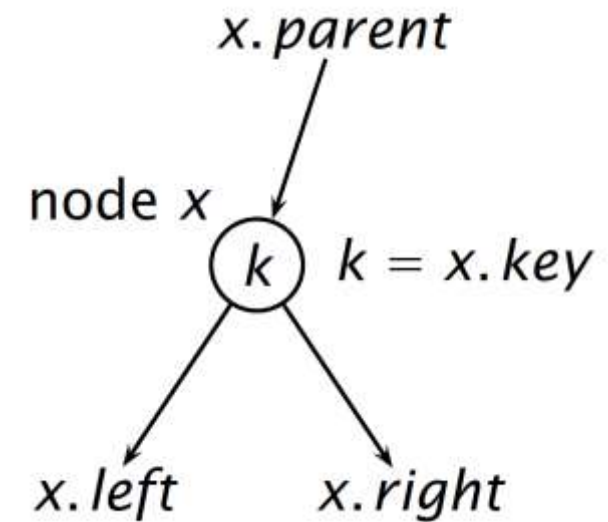
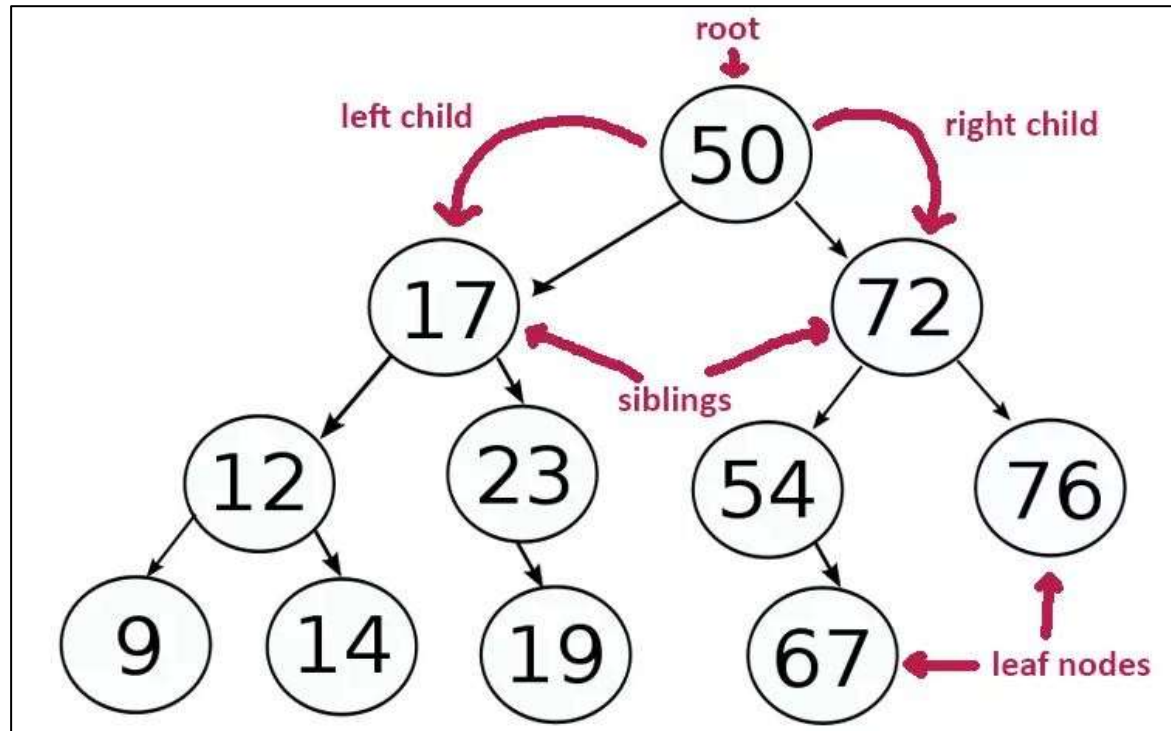
Implémentation d'un dictionnaire : Liste chaînée

- Insertion (resp. suppression):
 - Il faut chercher la clé dans la liste
 - Si elle existe, on remplace la valeur (resp. on la supprime)
 - Si elle n'existe pas, on la place dans la tête de la liste (resp. on ne fait rien)
- Recherche : voir algorithme ci-contre

```
LIST-SEARCH( $L, k$ )  
1   $x = L.head$   
2  while  $x \neq \text{NIL} \wedge x.key \neq k$   
3       $x = x.next$   
4  return  $x$ 
```

Implémentation d'un dictionnaire avec BST:

Arbre BST (ABR)



Implémentation d'un dictionnaire avec BST:

insert(k, v)

```
2 from src.adt.map.BinaryNode import Node
3 from src.adt.map.Entry import Entry
4
5
6 class BsTDict(object):
7
8     def __init__(self):
9         self.root: Node = None
```

```
12 def insert(self, key, value):
13     if self.root is None:
14         self.root = Node(key, value)
15     else:
16         self.attach(self.root, Node(key, value))
17
18 def attach(self, root, new_node):
19     if root is not None:
20         if root.entry.key <= new_node.entry.key:
21             if root.left_child:
22                 self.attach(root.left_child, new_node)
23             else:
24                 root.left_child = new_node
25         elif root.entry.key > new_node.entry.key:
26             if root.right_child:
27                 self.attach(root.right_child, new_node)
28             else:
29                 root.right_child = new_node
```

Implémentation d'un dictionnaire avec BST: get(k)

```
1 package colval.adt
2
3 class BSTMap[K,V] (compare:(K,K)=>Int) extends scala.collection.mutable.Map[K,V] {
4
5     class Node(var key: K, var value: V, var left: Node, var right: Node)
6     private var root: Node = _
7
8     def get(key: K): Option[V] = {
9         var rover = root
10        var c = if (rover != null) compare(key, rover.key) else 0
11        while (rover != null && c != 0) {
12            rover = if (c < 0) rover.left else rover.right
13            c = if (rover != null) compare(key, rover.key) else 0
14        }
15        if (rover == null) None else Some(rover.value)
16    }
```

Implémentation d'un dictionnaire :

Comparaison de complexité des différentes implémentations

- Dans les conditions extrêmes, ex. BST non balancé.
- S'il est balancé, la profondeur de BST est égale à $\log n$. Donc la complexité attendue de l'opération d'insertion est en $O(\log n)$

Opérations	Tableau non trié (keys)	Tableau trié (toujours)	Un tableau à accès direct	Liste chaînée Simple + head	BST non balancé
put(k,v)	$O(1)$	$O(n) > \text{shift}$	$O(1)$ k=index	$O(n)$	$O(n)$
get(k)	$O(n)$	$O(\log n)$	$O(1)$ k=index	$O(n)$	$O(n)$
delete(k)	$O(n) > \text{shift}$	$O(n) > \text{shift}$	$O(1)$	$O(n) > \text{find}$	$O(n)$
find(k)	$O(n)$	$O(\log n)$	$O(1)$	$O(n)$	$O(n)$
Is_empty()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
length()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Références

- <https://opensa-server.cs.vt.edu/ODSA/Books/CS3/html/Dictionary.html>
- <https://slideplayer.com/slide/4560099/>
- <https://gist.github.com/ourway/7fa9aefce098d6377a6d>